

Charlie (not a person in a dog costume) (Forensics)

Description

We're provided with an inconspicuous image of a dog and know that this is a forensics challenge. So, there must be some data hidden within the image.

Exif?

We can first try to use an exif tool to see if any useful data is in the metadata.

There's online tools like <https://exif.tools/> and both Mac OS and Windows comes with tools to look at photo metadata.

We'll be using exif tool in Kali Linux.

```
(kali㉿kali)-[~]  
$ exiftool charlie.jpg  
ExifTool Version Number      : 13.10  
File Name                     : charlie.jpg  
Directory                    : .  
File Size                     : 4.5 MB  
File Modification Date/Time   : 2025:05:24 12:49:47-07:00  
File Access Date/Time        : 2025:05:24 12:50:57-07:00  
File Inode Change Date/Time   : 2025:05:24 12:50:47-07:00  
File Permissions              : -rw-rw-r--  
File Type                     : JPEG  
File Type Extension          : jpg  
MIME Type                     : image/jpeg  
Exif Byte Order               : Big-endian (Motorola, MM)  
Image Width                   : 3024  
Image Height                  : 4032  
Encoding Process              : Baseline DCT, Huffman coding  
Bits Per Sample               : 8  
Color Components              : 3  
Y Cb Cr Sub Sampling         : YCbCr4:2:0 (2 2)  
Image Size                    : 3024x4032  
Megapixels                    : 12.2
```

There doesn't appear to be any useful information here though.

Binwalk

Next, we'll attempt to see if there's any embedded files in the image using Binwalk.

The command `binwalk -e filename` is used to analyze a binary file and automatically extract any embedded files or data that it detects within that file.

```
(kali㉿kali)-[~]
$ binwalk -e charlie.jpg
```

DECIMAL	HEXADECIMAL	DESCRIPTION
4477378	0x4451C2	Zip archive data, at least v2.0 to extract, compressed size: 16938, uncompressed size: 70636, name: flag.txt

It looks like there's a zip archive with a file, `flag.txt`.

Binwalk creates a directory to store extracted data and we can find `flag.txt` in it.

```
(kali㉿kali)-[~]  
$ cd _charlie.jpg.extracted  
  
(kali㉿kali)-[~/_charlie.jpg.extracted]  
$ ls  
4451C2.zip  flag.txt
```

After opening the file, there appears to just be a bunch of random text in it.

[illegible]

This looks very similar to base64, which is a binary-to-text encoding scheme that converts binary data into a string of printable ASCII characters.

We can use any online or command line base64 decoder to get the original data, but [CyberChef](#) is a great tool for this.

The screenshot shows the CyberChef web application interface. On the left, the 'Recipe' panel is set to 'From Base64' with the alphabet 'A-Za-z0-9+/' and options for 'Remove non-alphabet chars' (checked) and 'Strict mode' (unchecked). The 'Input' panel contains a long Base64-encoded string. The 'Output' panel displays the decoded result, which begins with the JFIF header of a JPEG image. The 'File details' panel on the right shows the file is named 'flag.txt', is 70,636 bytes in size, and is of type 'application/octet-stream'. The bottom of the interface features a 'STEP' button, a green 'BAKE!' button, and an 'Auto Bake' checkbox.

This results in what also looks like gibberish, however, the JFIF in the beginning hints that this might be a JPEG file. Thankfully, CyberChef also allows us to render images.

Recipe

From Base64

Alphabet
A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

Render Image

Input format
Raw

STEP

BAKE!

☒ Auto Bake

Input

File details



Name: flag.txt

Size: 70,636 bytes

Type: application/octet-stream

Loaded: 100%

Output

SVBGR{B1NW4LK_F7N}

We have the flag!

SVBGR{B1NW4LK_F7N}